Davante

12

BASE DE DATOS

Opciones para el tratamiento de los datos

ÍNDICE

/ 1. Introducción y contextualización práctica 4

/ 2. Inserción de registros 5

/ 3. Modificación de registros 6

3.1. Sintaxis 6

3.2. Modificación mediante datos introducidos por el usuario 6

3.3. Modificación de datos mediante subconsulta 6

/ 4. Eliminación de registros 7

4.1. Sintaxis 7

4.2. Eliminación de datos mediante datos introducidos por el usuario 7

4.3. Eliminación de datos mediante subconsulta 7

/ 5. Integridad referencial en la inserción, modificación y eliminación de datos 8

5.1. Integridad referencial en la inserción de datos 8

5.2. Integridad referencial en la modificación de datos 8

5.3. Integridad referencial en la eliminación de datos 8

/ 6. Consultas al diccionario de datos 9

/ 7. Extensiones del lenguaje: Almacenamiento de XML 10

/ 8. Extensiones del lenguaje: Almacenamiento de JSON 11

© MEDAC ISBN: 978-84-19885-68-5
Reservados todos los derechos. Queda rigurosamente prohibida, sin la autorización escrita de los titulares del copyright, bajo las sanciones establecidas en las leyes, la reproducción, transmisión y distribución total o parcial de esta obra por cualquier medio o procedimiento, incluidos la reprografía y el tratamiento informático.

ÍNDICE

/ 9. Extensiones del lenguaje: Conjuntos de resultados avanzados Pivot y Unpivot 12

9.1. Pivot 12

9.2. Unpivot 13

/ 10. Extensiones del lenguaje: Las funciones de agregación avanzadas ROLLUP, CUBE y GROUPING SETS 13

10.1. ROLLUP 13

10.2. CUBE 13

10.3. GROUPING SETS 14

/ 11. Herramientas para la optimización de consultas 14

/ 12. Caso práctico 1: “Eliminar empleados con faltas graves” 15

/ 13. Caso práctico 2: “Consultas sobre PACIENTES” 16

/ 14. Resumen y resolución del caso práctico de la unidad 17

/ 15. Bibliografía 18

OBJETIVOS

Expandir el análisis de los comandos y las cláusulas del lenguaje de manipulación de datos (DML).

Reconocer y redactar sentencias para la inserción, modificación y eliminación de registros manteniendo la integridad referencial.

Conocer la forma de consultar el diccionario de datos.

Explorar las distintas extensiones del lenguaje.

Analizar las herramientas de optimización de consultas.

/ 1. Introducción y contextualización práctica

El correcto manejo de la información en una base de datos es fundamental para cualquier sistema. Al hablar del tratamiento de datos, nos referimos en términos generales a los procesos de modificación, eliminación o inserción de datos.



Fig.1. Es posible insertar, modificar o eliminar datos de una base de datos mediante sentencias SQL.

En temas anteriores nos hemos centrado en diversas formas y opciones para acceder a la información, en este, exploraremos cómo se procesan esos datos y las opciones disponibles para ello. Analizaremos los comandos y las cláusulas que nos permiten construir sentencias SQL para llevar a cabo estas tareas, así como la manera de mantener la integridad referencial cuando se ejecutan dichas sentencias.

Además, examinaremos las diferentes extensiones del lenguaje SQL y aprenderemos a consultar el diccionario de datos y las herramientas para optimizar las consultas a la base de datos.

A continuación, vamos a plantear un caso práctico a través del cual podremos aproximarnos de forma práctica a la teoría de este tema.

Escucha el siguiente audio donde planteamos la contextualización práctica de este tema. Encontrarás su resolución en el apartado 'Resumen y resolución del caso práctico'.

Audio intro. "Modificando información de una base de datos"

https://on.soundcloud.com/CTvo717DQhAjiPYX6

/ 2. Inserción de registros

El tratamiento de datos puede realizarse usando comandos y sentencias SQL o utilizando herramientas gráficas específicas de cada sistema de gestión de bases de datos, dependiendo del sistema en uso. La elección entre uno u otro método dependerá de las necesidades particulares en cada situación.

Para saber más sobre las herramientas gráficas, escucharemos el siguiente audio:



Audio 1. "Herramientas gráficas para la edición de la información"
https://bit.ly/2Y0r7Qt



Para insertar nuevos registros en la BD, se emplea el comando **INSERT**. En la siguiente imagen se muestra su **sintaxis**:

```
INSERT INTO {Tabla | Vista} [(columna1 [,columna2]…) ]
VALUES (valor1[, valor2],…);
```

Fig.2. Sintaxis del comando INSERT.

Si los valores proporcionados corresponden a cada campo de la tabla y en el orden adecuado, se puede omitir la lista de columnas. Además, si no se especifica el valor de un campo, dicho campo tomará el valor NULL, siempre que sea permitido.

En las siguientes imágenes se crea la tabla EMPLEADOS y se insertan nuevos registros en ella. En la última sentencia, se muestra de qué manera se puede insertar únicamente el valor para ciertos campos.

```
CREATE TABLE EMPLEADOS (
  ID_EMP    NUMBER(4)    PRIMARY KEY,
  NOMBRE    VARCHAR2(50) NOT NULL,
  FECHA_NAC  DATE);
```

Fig.3. Sentencia SQL para crear tabla EMPLEADOS.

```
INSERT INTO EMPLEADOS VALUES (1, 'Pepe', '01/01/1970');
INSERT INTO EMPLEADOS VALUES (2, 'Juan', NULL);
INSERT INTO EMPLEADOS(NOMBRE, ID_EMP) VALUES ('Ana', 5);
```

Fig.4. Ejemplos de inserción de nuevos registros en la tabla EMPLEADOS.

También es posible **insertar nuevos registros utilizando los resultados extraídos de una consulta**. En la siguiente imagen se muestra un ejemplo de ello.

```
INSERT INTO EMPLEADOS
SELECT ID_NUM, NOMBRE, NACIMIENTO
FROM DEMANDANTES
WHERE ESTUDIOS='ASIR';
```

Fig.5. Ejemplo de inserción de registros a través de consultas SELECT.

/ 3. Modificación de registros

El comando **UPDATE** permite **modificar o actualizar los valores de uno o varios registros en una tabla o vista de la base de datos**.

3.1. Sintaxis

La cláusula UPDATE tiene la siguiente **sintaxis**:

```
UPDATE {Tabla | Vista}
SET columna1 = valor1 [, columna2 = valor2]…
[WHERE Condición];
```

Fig.6. Sintaxis de sentencia UPDATE.

En la cláusula **SET** se **especifican las columnas que se desea modificar junto con los nuevos valores**. La sentencia **WHERE es opcional**, pero suele ser muy útil para **definir las condiciones necesarias para la modificación de registros**, permitiendo así excluir aquellos que no queremos cambiar.

3.2. Modificación mediante datos introducidos por el usuario

En la siguiente imagen se muestra un ejemplo en el que se lleva a cabo la modificación de los campos 'localidad' y 'nombre' de todos los empleados con fecha de nacimiento posterior al 01/12/1980. En este caso, la modificación es para que comiencen con mayúscula. **Solo se modificarán las columnas especificadas en la cláusula SET que cumplan la condición del WHERE**.

```
UPDATE EMPLEADOS
SET NOMBRE=INITCAP(NOMBRE), LOCALIDAD='Madrid'
WHERE FECHA_NAC >= '01/12/1980';
```

Fig.7. Ejemplo de sentencia SQL utilizando UPDATE.

3.3. Modificación de datos mediante subconsulta

Se pueden modificar datos utilizando subconsultas como condición en la cláusula WHERE. En la siguiente imagen se muestra un ejemplo en el que se incrementa un 15% el salario de los empleados que pertenecen a la sección denominada 'Productivo' en la tabla 'secciones'.

```
UPDATE EMPLEADOS
SET salario=salario*1.15
WHERE id_seccion = (SELECT id_seccion FROM secciones
                    WHERE nom_seccion='Productivo');
```

Fig.8. Ejemplo de modificación de datos mediante subconsulta.

Enlaces de interés...

En este enlace encontrarás un vídeo en el que podrás ver algunos ejemplos prácticos de cómo realizar consultas utilizando el comando de modificación de registros UPDATE.

/ 4. Eliminación de registros

El comando **DELETE** permite **eliminar los registros específicos de una tabla basándose en una condición determinada**.

4.1. Sintaxis

En la siguiente imagen se muestra la **sintaxis de la sentencia DELETE**:

```
DELETE [FROM] {Tabla | Vista}
[WHERE Condición];
```

Fig.9. Sintaxis de la sentencia DELETE.

Igual que en la sentencia UPDATE, la cláusula **WHERE es opcional**, pero generalmente resulta muy útil para **definir las condiciones necesarias para la eliminación de registros**.



Vídeo 1. "Uso de herramientas gráficas para la gestión de la información"

https://bit.ly/3kOQtL1



4.2. Eliminación de datos mediante datos introducidos por el usuario

Borrar un registro es una operación delicada que no debe comprometer la integridad de la base de datos. Por esta razón, es conveniente **realizar una copia de seguridad antes de proceder** y solo llevar a cabo la eliminación si se tiene total certeza de su necesidad.

```
-- Se borran los empleados de Madrid
DELETE EMPLEADOS
WHERE LOCALIDAD='Madrid';

-- Se borran empleados de Ciudad Real
-- cuya fecha de nacimiento sea anterior a 1980

DELETE EMPLEADOS
WHERE FECHA_NAC < '01/01/1980' AND LOCALIDAD = 'C.Real';
```

Fig.10. Ejemplos de uso del comando DELETE.

4.3. Eliminación de datos mediante subconsulta

Existe la posibilidad de **utilizar subconsultas como condición en la cláusula WHERE para borrar datos**, igual que ocurre con los comandos INSERT, SELECT o UPDATE.

```
DELETE EMPLEADOS
WHERE id_empleado IN
(SELECT id_empleado FROM OPERARIOS);
```

Fig.11. Ejemplo del uso del comando DELETE con WHERE.

En el ejemplo previo de eliminación de datos mediante subconsulta, se eliminarán los empleados que estén clasificados como operarios.

Enlaces de interés...

En este enlace encontrarás un vídeo en el que podrás ver algunos ejemplos prácticos de cómo realizar consultas utilizando el comando de eliminación de registros DELETE.

5. Integridad referencial en la inserción, modificación y eliminación de datos

Como hemos visto en temas anteriores, existen múltiples acciones a realizar en la gestión de la integridad referencial durante las operaciones de manipulación de datos.

5.1. Integridad referencial en la inserción de datos

Al insertar datos en una tabla con integridad referencial (es decir, una tabla que contiene una clave foránea), **la integridad referencial asegura que el valor de esta clave foránea exista previamente en la tabla referenciada** (la que contiene la clave primaria correspondiente). **Si el valor no existe en la tabla referenciada, se producirá un error y la inserción no se realizará**. La siguiente imagen muestra un ejemplo del error mostrado en Oracle SQL Developer al intentar insertar un elemento que no cumple con la integridad referencial.

```
Informe de error -
ORA-00001: restricción única (SYS.PK_EMP) violada
```

Fig.12. Ejemplo de error de integridad.

5.2. Integridad referencial en la modificación de datos

Al modificar la clave primaria en la tabla principal o la clave foránea en la tabla secundaria, **la integridad referencial asegura que la actualización sea válida para todos los registros relacionados**, de acuerdo con la acción configurada en la restricción (RESTRICT, CASCADE, SET NULL, SET DEFAULT). Cuando se modifica una clave foránea, hay que verificar que el nuevo valor sea clave primaria en la tabla principal.

5.3. Integridad referencial en la eliminación de datos

Cuando se elimina un registro en la tabla principal, **las reglas establecidas por las claves foráneas en las tablas relacionadas decidirán qué sucederá con los registros que estaban vinculados** al registro eliminado.

Como ejemplo, supongamos que, en una tabla de pedidos, cada uno de estos está vinculado a un cliente y eliminamos a un cliente. El comportamiento para cada posible acción de este ejemplo se muestra en la siguiente tabla:

Acción	Resultado
Restrict	Impide que se elimine el cliente.
Cascade	Elimina todos los pedidos vinculados al cliente.
Set Null	Establecería con valor NULL la referencia al cliente en los pedidos del cliente eliminado.
Set Default	Establecería el valor por defecto según el tipo de dato del campo que referencia al cliente.

Tabla 1. Resultado de eliminación de cliente según cada acción de integridad referencial.

/ 6. Consultas al diccionario de datos

Como hemos visto en el tema titulado 'Fundamentos de las bases de datos', el diccionario de datos es la estructura en la que se almacenan todos los elementos que componen la base de datos: tablas, columnas, relaciones, etc.

En SQL, y específicamente en Oracle, es posible obtener esta información. Para hacerlo, podemos **consultar las siguientes vistas proporcionadas por el sistema**:

- **USER_TABLES y USER_TAB_COLUMNS**: Contienen la información sobre las tablas y las columnas de las tablas del usuario, respectivamente.
- **ALL_TABLES y ALL_TAB_COLUMNS**: Son similares a las anteriores, pero mostrando todos los objetos a los que el usuario tiene acceso en base a sus permisos y roles.
- **DBA_TABLES y DBA_TAB_COLUMNS**: Contienen la información de todos los objetos en la base de datos. Solo son accesibles por usuarios que tengan privilegios de Administrador (DBA).

En la siguiente imagen se muestra un ejemplo de consulta para comprobar las tablas a las que un usuario tiene acceso en SQL Live.

```
SELECT *
FROM user_tables;
```

Fig.13. Ejemplo de consulta de las tablas del usuario en SQL Live.

En la próxima imagen se ofrece el resultado de esta consulta.

TABLE_NAME	TABLESPACE_NAME	CLUSTER_NAME	IOT_NAME	STATUS	PCT_FREE
DEPT	LIVESQL_USERS	-	-	VALID	10
EMP	LIVESQL_USERS	-	-	VALID	10

Fig.14. Resultado del ejemplo de la consulta de las tablas del usuario en SQL Live.

En esta otra imagen se muestra un ejemplo de la consulta sobre la tabla del sistema 'user_tables' que almacena columnas, para obtener las columnas que componen la tabla DEPT.

```
SELECT tables.table_name, columns.column_name, columns.data_type, columns.data_length
FROM user_tables tables
INNER JOIN user_tab_columns columns ON tables.table_name = columns.table_name;
ORDER BY TABLE_NAME ASC, COLUMN_NAME ASC;
```

Fig.15. Consulta a tabla de columnas.

En la siguiente imagen se puede ver el resultado de la consulta.

TABLE_NAME	COLUMN_NAME	DATA_TYPE	DATA_LENGTH
DEPT	DEPTNO	NUMBER	22
DEPT	DNAME	VARCHAR2	14
DEPT	LOC	VARCHAR2	13
EMP	EMPNO	NUMBER	22
EMP	ENAME	VARCHAR2	10

Fig.16. Fragmento del resultado de la consulta.

/ 7. Extensiones del lenguaje: Almacenamiento de XML

En Oracle, al igual que en otros SGBD que se basan en el estándar SQL, **es posible almacenar y gestionar datos en formato XML**. En las siguientes imágenes se muestran los **pasos necesarios para crear una tabla que almacene datos XML y las consultas necesarias para insertarlos, filtrarlos y extraerlos**.

```
CREATE TABLE ficha_producto (
    id_producto NUMBER,
    xml_col XMLType
);
```

Fig.17. Creación de tabla con XML.

```
INSERT INTO ficha_producto (id_producto, xml_col)
VALUES (
    1,
    XMLType(
    '<?xml version="1.0"?>
    <producto>
        <nombre>Laptop</nombre>
        <precio>1200.00</precio>
        <caracteristicas>
            <procesador>AMD Ryzen 7</procesador>
            <ram>16GB</ram>
            <almacenamiento>512GB SSD</almacenamiento>
        </caracteristicas>
    </producto>'
    )
);
```

Fig.18. Inserción en columna XML.

```
SELECT id_producto, xml_col
FROM ficha_producto
WHERE id_producto = 1;
```

Fig.19. Consulta en tabla con XML.

```
SELECT EXTRACTVALUE(xml_col, '/producto/nombre/text()')
       AS procesador_producto
FROM ficha_producto
WHERE XMLExists('/producto/caracteristicas/procesador[text()="AMD Ryzen 7"]'
                PASSING xml_col);
```

Fig.20. Extracción y filtrado de XML.

Las **funciones más comunes para el tratamiento de XML** se muestran en la siguiente tabla:

Función	Descripción
XMLTable	Convierte fragmentos XML en filas y columnas.
XMLQuery	Evalúa una expresión XQuery (instrucción de consulta Xml) y devuelve un resultado XMLType.
XMLExists	Verifica si una expresión XQuery devuelve resultados verdaderos (booleanos).
XMLCast	Convierte XML a tipos de datos SQL estándar.
XMLSerialize	Convierte XMLType (tipo de dato de Oracle para datos XML) a una cadena CLOB o VARCHAR.
extractValue	Extrae el valor de un nodo XML especificado como una cadena (obsoleta en versiones recientes).

Tabla 2. Funciones XML más habituales.

Investigamos...

Además de las funciones vistas para el tratamiento de datos XML, existen otras como LOAD DATA o HTTPURITYPE que permiten insertar datos XML desde fuentes externas. Utiliza Google Scholar para investigar acerca de su uso.

/ 8. Extensiones del lenguaje: Almacenamiento de JSON

En el punto anterior hemos visto cómo almacenar y gestionar datos en formato XML. De igual modo, **en Oracle es posible almacenar y gestionar datos en formato JSON**. Para ello disponemos de las **funciones** mostradas en la siguiente tabla, **que nos facilitan la tarea**.

Función	Descripción
JSON_VALUE	Extrae un valor escalar de un documento JSON.
JSON_QUERY	Extrae un objeto o array JSON.
JSON_TABLE	Convierte datos JSON en filas y columnas SQL.
HTTPURITYPE	Permite leer datos desde una URL externa.
UTL_HTTP	Permite realizar solicitudes HTTP desde PL/SQL.

Tabla 3. Funciones para el tratamiento de datos JSON.

En las siguientes imágenes se muestran los **pasos necesarios para realizar la inserción, selección y filtrado de datos de tipo JSON**.

```
CREATE TABLE datos_producto (
    id_producto NUMBER,
    json_col CLOB CHECK (json_col IS JSON)
);
```

Fig.21. Creación de tabla con JSON.

```
INSERT INTO datos_producto (id_producto, json_col)
VALUES (
    1,
    '{
        "producto": {
            "nombre": "Laptop",
            "precio": 1200.00,
            "caracteristicas": {
                "procesador": "AMD Ryzen 7",
                "ram": "16GB",
                "almacenamiento": "512GB SSD"
            }
        }
    }'
);
```

Fig.22. Inserción en columna JSON.

```
SELECT JSON_VALUE(json_col, '$.producto.nombre')
    AS nombre_producto
FROM datos_producto
WHERE id_producto = 1;
```

Fig.23. Extracción de datos en JSON.

```
SELECT JSON_VALUE(json_col, '$.producto.nombre')
    AS nombre_producto
FROM datos_producto
WHERE JSON_VALUE(json_col, '$.producto.caracteristicas.procesador')
        = 'AMD Ryzen 7';
```

Fig.24. Extracción y filtrado de JSON.

/ 9. Extensiones del lenguaje: Conjuntos de resultados avanzados Pivot y Unpivot

Pivot y Unpivot son funciones que **permiten intercambiar las filas y columnas de un resultado para ofrecer una vista reorganizada de los datos** almacenados en las tablas de la base de datos.

9.1. Pivot

Esta función transforma las filas en columnas, reorganizando los datos y permitiendo realizar agregaciones que faciliten la lectura y el análisis de los mismos.

En las siguientes imágenes se muestra un ejemplo del uso de Pivot para realizar una agrupación de las ventas trimestrales:

	AÑO	TRIMESTRE	VENTAS
1	2022	1	1000
2	2022	2	1500
3	2022	3	2000
4	2022	4	2500
5	2023	1	1100
6	2023	2	1600
7	2023	3	2100
8	2023	4	2600

Fig.25. Ventas trimestrales.

Como se puede ver, se obtiene un listado con las ventas separadas por cada año y trimestre en una fila diferente. Esto podría resultar difícil de analizar si tuviésemos en cuenta las ventas de más años.

La sentencia SQL de la siguiente imagen nos ayudará a mejorar esta visualización.

```
SELECT *
FROM (
    SELECT año, trimestre, ventas
    FROM ventas
)
PIVOT (
    SUM(ventas) FOR trimestre
    IN (1 AS Q1, 2 AS Q2, 3 AS Q3, 4 AS Q4)
)
ORDER BY año;
```

Fig.26. Ejemplo de uso de Pivot.

En la siguiente imagen, se muestra el resultado de la consulta de la imagen anterior. En ella podemos ver las ventas trimestrales de una forma más organizada y fácil de analizar.

AÑO	Q1	Q2	Q3	Q4
2022	1000	1500	2000	2500
2023	1100	1600	2100	2600

Fig.27. Resultado del uso de Pivot.

9.2. Unpivot

Esta función lleva a cabo la transformación de columnas en filas, es decir, la opuesta a Pivot. En la siguiente imagen se muestra la consulta que haría la transformación inversa.

```
SELECT año, trimestre, ventas
FROM ventasQ
UNPIVOT (
    ventas FOR trimestre IN (Q1 AS 1, Q2 AS 2, Q3 AS 3, Q4 AS 4)
)
ORDER BY año, trimestre;
```

Fig.28. Ejemplo de sentencia Unpivot.



Audio 2. "Consultas jerárquicas"
https://on.soundcloud.com/pwP5bYezQSd1QgxL8



/ 10. Extensiones del lenguaje: Las funciones de agregación avanzadas ROLLUP, CUBE y GROUPING SETS

Las extensiones del lenguaje SQL, como las funciones de agregación avanzadas **ROLLUP**, **CUBE** y **GROUPING SETS**, **se utilizan para resumir y analizar datos de forma más detallada**. Simplifican el análisis de datos.

10.1. ROLLUP

Permite **generar subtotales y un total general en una sola consulta** al agregar filas en una jerarquía concreta. Es una **extensión de GROUP BY** que **facilita la creación de resúmenes de datos**.

```
SELECT departamento, producto, SUM(ventas)
FROM ventas
GROUP BY ROLLUP (departamento, producto);
```

Fig.29. Ejemplo de sentencia con ROLLUP.

En el ejemplo de la imagen anterior, se obtendrán subtotales para cada departamento y un total general para todas las ventas.

10.2. CUBE

La función **CUBE**, que también es una **extensión de GROUP BY**, **genera subtotales para todas las combinaciones posibles de las columnas indicadas, incluyendo los totales generales**. A diferencia de ROLLUP, que sigue una jerarquía, CUBE **crea un cubo de datos con todas las permutaciones de las columnas**.

```
SELECT departamento, producto, SUM(ventas)
FROM ventas
GROUP BY CUBE (departamento, producto);
```

Fig.30. Ejemplo de sentencia con CUBE.

En el ejemplo de la imagen anterior, se generarán subtotales para cada combinación de departamento y producto, además de un total general para todas las ventas.

10.3. GROUPING SETS

Define específicamente múltiples agrupaciones en una sola consulta. Esto ofrece una gran flexibilidad, ya que especifica exactamente los grupos para los que se desean subtotales.

```
SELECT departamento, producto, SUM(ventas)
FROM ventas
GROUP BY GROUPING SETS ((departamento, producto),
                        (departamento),(producto),());
```

Fig.31. Ejemplo de sentencia con GROUPING SETS.

En la imagen anterior, se muestra un ejemplo que produce subtotales para cada combinación de departamento y producto, solo por departamento, solo por producto y un total general.

Enlaces de interés...

En el siguiente vídeo, encontrarás ejemplos del uso de consultas jerárquicas, que permiten extraer información de una base de datos organizada jerárquicamente, como en el caso de un organigrama empresarial: https://www.youtube.com/watch?v=8IvRVChnChY

/ 11. Herramientas para la optimización de consultas

SQL permite optimizar consultas con las siguientes herramientas:

- **Plan de ejecución**: Permite **visualizar los pasos que el motor de la base de datos seguirá para ejecutar la consulta**, como se ve en la siguiente imagen:

```
EXPLAIN PLAN FOR
SELECT * FROM empleados WHERE departamento_id = 1;

SELECT * FROM TABLE(DBMS_XPLAN.DISPLAY);
```

Fig.32. Consulta del plan de ejecución.

- **Sugerencias para el optimizador**: **Hacen posible dar indicaciones al optimizador de consultas**, por ejemplo, indicándole qué índice utilizar. Se especifican con el formato **/*+ Sugerencia */**. En la siguiente imagen se muestra un ejemplo:

```
SELECT /*+ INDEX(empleados emp_depto_idx) */ *
FROM empleados
WHERE departamento_id = 1;
```

Fig.33. Sugerencias del optimizador.

- **Particionamiento**: Permite **dividir una tabla grande en partes más pequeñas** y manejables, mejorando así el rendimiento de las consultas.

```
PARTITION BY RANGE (fecha_contratacion) (
  PARTITION p1 VALUES LESS THAN (TO_DATE('2022-01-01', 'YYYY-MM-DD')),
  PARTITION p2 VALUES LESS THAN (TO_DATE('2023-01-01', 'YYYY-MM-DD')),
  PARTITION p3 VALUES LESS THAN (MAXVALUE)
);
```

Fig.34. Ejemplo de particionamiento.

Investigamos...

Además de las herramientas de optimización de consultas mostradas aquí, los SGBD ofrecen opciones como Dynamic Sampling, Database Replay, SQL Performance Analyzer, AWR, ADDM, TuningAdvisor y SQL Dinámico. Investiga en Google Scholar sus ventajas y casos de uso.

- **Vistas materializadas**: Almacenan los resultados de una consulta y pueden actualizarse periódicamente. Son **muy útiles ante consultas complejas y costosas**. Se muestra un ejemplo de ello en la siguiente imagen:

```
CREATE MATERIALIZED VIEW emp_salario_depto
AS
SELECT departamento_id, AVG(salario) avg_salario
FROM empleados
GROUP BY departamento_id;
```

Fig.35. Ejemplo de vistas materializadas.

- **Perfiles de ejecución SQL**: Los perfiles de ejecución SQL permiten **optimizar el plan de ejecución de las consultas** al almacenar información sobre las recomendaciones calculadas en ejecuciones previas.

- **Variables enlazadas**: Si deseamos ejecutar una consulta varias veces, pero con diferentes valores en un campo del WHERE en cada ejecución, esto impediría la reutilización del Perfil SQL, ya que la consulta sería distinta cada vez. **Las variables enlazadas permiten que dos consultas con valores diferentes se traten como la misma consulta**. En la siguiente imagen se muestra un ejemplo utilizando la variable ':dept_id'. Esto es muy útil en consultas complejas con varios *joins* y subconsultas.

```
SELECT * FROM empleados
WHERE departamento_id = :dept_id;
```

Fig.36. Ejemplo de variables enlazadas.

/ 12. Caso práctico 1: "Eliminar empleados con faltas graves"

Planteamiento: Nuestro amigo de la tienda de informática nos pide ayuda para elaborar una sentencia SQL que le permita manejar cierta información de su base de datos. Concretamente, necesita poder eliminar de la tabla EMPLEADOS aquellos empleados que tengan faltas graves, las cuales estarán registradas en la tabla FALTAS_GRAVES. Ambas tablas tienen una columna llamada 'id_empleado' a través de la cual se relacionan.

Id_empleado	Nombre	Apellidos	Domicilio	Domicilio

Tabla 4. Tabla EMPLEADOS.

Id_empleado	Fecha	Descripción

Tabla 5. Tabla FALTAS_GRAVES.

Nudo: ¿Qué sentencia SQL podemos facilitarle para eliminar esta información?

Desenlace: Apoyándonos en los conocimientos adquiridos durante este y anteriores temas, consideramos que la siguiente consulta serviría para sus necesidades.

```
DELETE EMPLEADOS
WHERE ID_EMPLEADO IN (
    SELECT ID_EMPLEADO
    FROM FALTAS_GRAVES
);
```

Fig.37. Sentencia de eliminación con subconsulta.

/ 13. Caso práctico 2: "Consultas sobre PACIENTES"

Planteamiento: Supongamos que en nuestra empresa se gestiona la base de datos de los hospitales de una determinada región. Esta base de datos está formada por varias tablas que almacenan la información. Una de ellas es la tabla PACIENTE, que tiene la siguiente estructura.

Id (NUMBER)	Nombre (VARCHAR)	Dirección (VARCHAR)	Fecha_NAC (DATE)	NSS (NUMBER)

Tabla 6. Columnas de la tabla PACIENTE.

Nudo: ¿Qué sentencias SQL serían necesarias para realizar las siguientes modificaciones?

1. Añadir un nuevo paciente, llamado J. García, con ID n.º 55, que vive en C/Algeciras, 5, que nació el 29/09/1986 y cuyo NSS es 123456789.

2. Modificar la dirección del paciente número 74835 a C/Coruña, 41.

3. Asignar al paciente número 240210 la dirección almacenada para el paciente número 270612.

Desenlace: Con los conocimientos adquiridos hasta ahora, hemos determinado que las consultas mostradas en las siguientes imágenes permitirían realizar las modificaciones solicitadas en la base de datos.

1. Añadir nuevo paciente:

```
INSERT INTO PACIENTE
VALUES (55,'J.Garcia','C/Algeciras 5','29/09/1986',123456789);
```

Fig.38. Inserción de paciente.

2. Modificar dirección:

```
UPDATE PACIENTE
SET dirección = 'C/Coruña 41'
WHERE id = 74835;
```

Fig.39. Actualización de paciente.

3. Asignar dirección de otro paciente:

```
UPDATE PACIENTE
SET dirección = (SELECT dirección FROM PACIENTE WHERE id = 270612)
WHERE id = 240210;
```

Fig.40. Actualización con el resultado de la subconsulta.

/ 14. Resumen y resolución del caso práctico de la unidad

En este tema hemos explorado las maneras de crear, modificar y eliminar registros, así como la gestión de la integridad referencial durante estas operaciones.

También hemos abordado la forma de efectuar consultas al diccionario de datos y las diversas extensiones del lenguaje SQL.

Finalmente, nos hemos adentrado en las herramientas disponibles para la optimización de consultas.

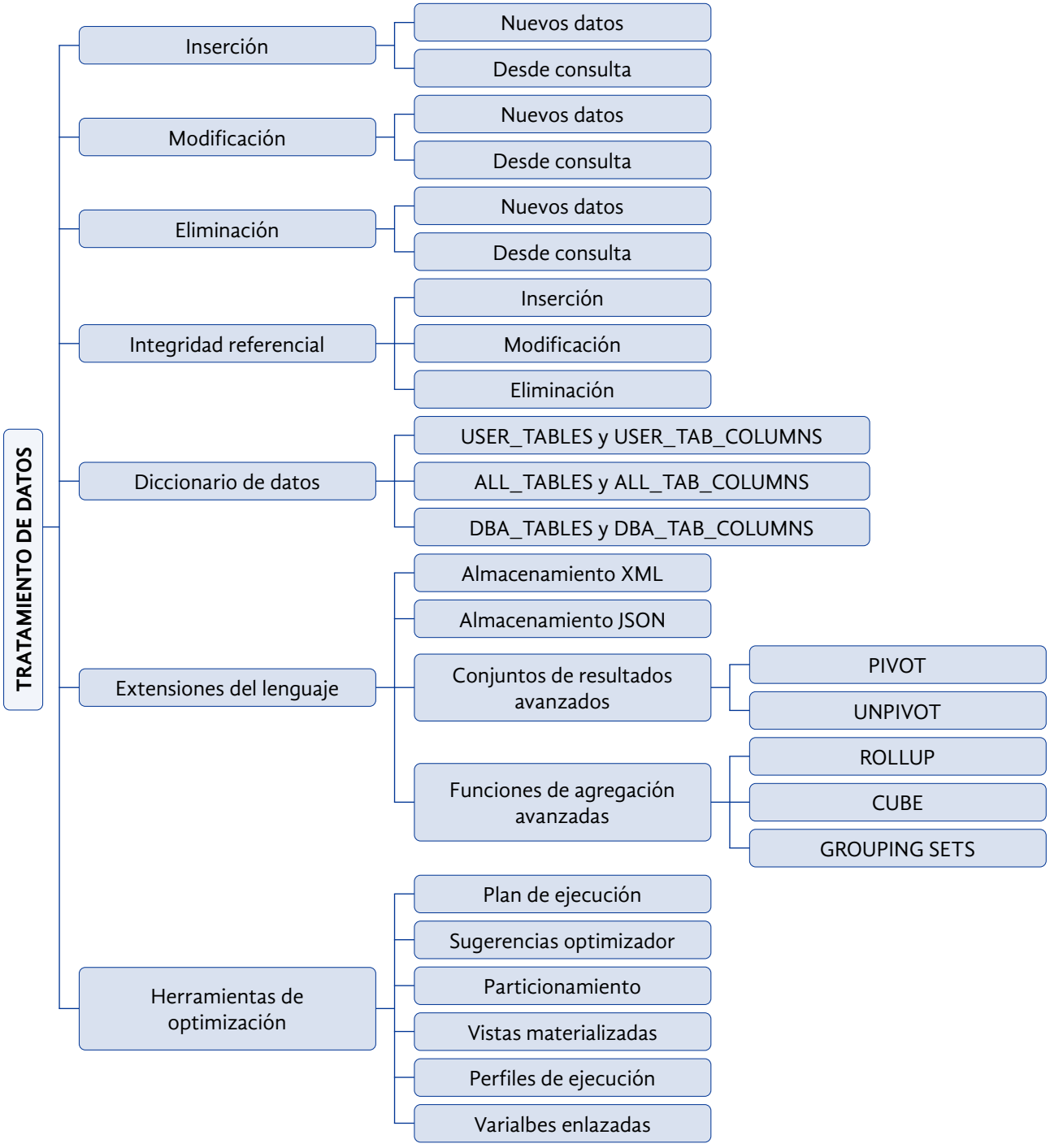


Fig.41. Esquema resumen del tema.

Resolución del caso práctico de la unidad

En el caso práctico de la introducción del tema, planteábamos la necesidad de nuestro amigo de dar de alta a una nueva empleada en la base de datos de su tienda de informática. En líneas generales, podremos ayudar a nuestro amigo a insertar, modificar y eliminar la información de la base de datos de su tienda mediante el uso de las herramientas gráficas que ofrece el SGBD, así como a través de las diferentes sentencias de inserción, modificación y eliminación que hemos visto en este tema.

Además, tras haber estudiado los diferentes comandos de SQL para la manipulación de datos, la siguiente consulta debería servir como solución a su necesidad:

```
INSERT INTO EMPLEADOS
VALUES  ('107', 'Ada', 'Rey Valencia',
         '29242426', 'AVDA DE LA RECONQUISTA 1');
```

Fig.42. Inserción de empleada.

/ 15. Bibliografía

Connolly, T. y Begg, C. (2005). *Sistemas de bases de datos* (5.ª ed.). Madrid, España. Addison-Wesley.

Elmasri, R. y Navathe, S. (2007). *Fundamentos de sistemas de bases de datos*. Addison-Wesley.

López Montalbán, Iván y De Castro, M. (2014). *Gestión de bases de datos: Técnico Superior en Administración de Sistemas Informáticos en Red*. Madrid. Garceta D.L.

Oppel, A. (2009). *Databases A Beginner's Guide*. McGraw-Hill Education.

Pérez Marqués, M. (2016). *Administración básica de bases de datos con Oracle 12c SQL*. Madrid, España. Alfaomega.

Sánchez, G. C. (2001). *Sistemas gestores de bases de datos*. Paraninfo.